

На вкус и цвет все фломастеры разные

Особенно если вы пишете на Ruby

Павел Аргентов · RubyRussia 2026

План

1. Хук: агент буксует на Ruby, а коллега на Python «закоммитил, поехали»
2. Жалоба превращается в фальсифицируемую гипотезу
3. Почему анекдотам верить нельзя: конфаундеры
4. Дизайн стенда: язык — единственная переменная
5. Честный стенд оказался труднее самого эксперимента
6. Первые наблюдения ($n = 1$ на ячейку — это не выводы)
7. Что дальше

0. Хук и постановка

Сцена, знакомая залу

- Агент на Ruby-тикете ходит по кругу: «нашёл придирку в саморевью — поправил — нашёл новую»
- Коллега на Python в соседнем канале: «закоммитил, поехали»
- Разворот: доклад **не про то, что Ruby хуже** — про то, как это проверить всерьёз
- Граница: сегодня диагноз. «Что делать на нелюбимом стеке» — отдельный доклад

1. Жалоба и гипотеза

Гипотеза

Агент неравнодушен к языку **не из-за качеств языка**, а из-за объёма кода этого языка в обучающей выборке модели. Два независимых механизма:

1. **Python** в фаворе — язык выбора самих авторов LLM
2. **JavaScript и TypeScript** — по массовости

Ruby не подпадает ни под один и по обеим причинам должен проигрывать.

«Объём кода в обучении» и «сколько народу пишет сейчас» — разные величины. Для Python, JS и Ruby совпадают по порядку; для TypeScript расходятся — вернёмся в секции 5.

Посылку подпирают внешние данные

Язык	The Stack v2	Токены	Stack Overflow 2025
JavaScript	~1115 ГБ	~45 млрд	—
Python	~233 ГБ	~50 млрд	—
TypeScript	~61 ГБ	~12 млрд	44–49 %
Ruby	~31.7 ГБ	~5.8 млрд	6–7 %

Ruby — вдвое меньше даже TypeScript, в 35 раз меньше JavaScript. RedMonk: №9, многолетнее снижение.

Это рыночный контекст, не результат эксперимента.

2. Почему анекдотам верить нельзя

Конфаундеры

- Разные проекты на разных языках несопоставимы по масштабу — разницу спишут на проект
- **Утечка обучающих данных**: модель могла запомнить фиксы к issue популярного репозитория
- Разные ассистенты и инструменты — разное поведение, не связанное с языком
- **Оператор и дрейф методики** — именно этот конфаундер нас потом и укусит

Вывод: нужен стенд, где язык — единственная переменная.

3. Дизайн эксперимента

Полигон

- Фиксированная связка: Claude Code headless (`claude -p`), модель `claude-sonnet-5` точным ID на каждом вызове, `--effort xhigh` явно
- Единственная переменная — язык проекта
- Языки доклада: **Python, JavaScript, TypeScript, Ruby**
- Go, Scala, OCaml — в запасе для другого исследования, в доклад не идут

Проект Syncbox

- Не готовый опенсорс, а **единая спецификация**, реализуемая с нуля на каждом языке
- HTTP API и CLI зафиксированы буквально, вплоть до имён флагов; внутреннее устройство свободно
- Одним ходом закрывает два конфаундера: сопоставимость проектов и утечку фиксов

```
# Пример: контракт CLI одинаков для всех семи языков
run_client("put", key: "note", body: "привет")    # => 201 Created
run_client("get", key: "note")                    # => 200 OK, "привет"
run_client("sync")                                # => разрешение конфликтов
```

Цикл на тикет

реализация

- саморевью на критические замечания
- штатные тесты + acceptance-смок + контрактный тест (Schemathesis по OpenAPI)
- фикс
- повторное ревью
- до сходимости или явного отказа

Три проверки — часть критерия «сошлось / сдалось», не постфактум.

Фиксируем на тикет: число итераций, время, токены — точными полями из JSON-вывода.

4. Честный стенд оказался труднее эксперимента

Как мы ловили сами себя

Что пробовали

- Файловая песочница через `--settings`
- Docker-обёртка вокруг вызова
- Проброшенный сокет Docker (DooD)

Где протекло

- Песочница держала только Bash, не Read/Write
- Сокет давал примонтировать любой хостовый путь
- Итог — Docker-in-Docker: хостовой ФС в принципе нет

Главный эпизод: Python, тикет 1 — агент нашёл эталонную реализацию через Bash+Read и почти дословно скопировал. Воспроизводилось на 100 %.

5. Первые наблюдения

Пилотная фаза — это не выводы

$n = 1$ **на ячейку**. 44 прогона: 4 языка × 11 тикетов.

Статистический тест на таких объёмах не осмыслен.

Что показал пилот

- По бинарному «агент справился» **эффекта языка нет**: 4/4 языка на 11/11 тикетов сошлись
- Цена успеха: Ruby **не дороже**, чем {Python ≈ JavaScript}, дальше TypeScript
- Анекдотический «Ruby хуже всех» **не воспроизвёлся** — под единой методикой Ruby самый дешёвый по деньгам и out-токенам
- Единственный Ruby-сигнал: медленный по календарю, но **не по времени модели** — разницу даёт работа вокруг модели (тесты, пересборка): ~21 мин против ~7 у Python

TypeScript и гипотеза

- TypeScript дороже всех — согласуется с тем, что его корпус в разы тоньше
- Гипотеза объединяла JS и TS в один «массовый» механизм — данные их разводят
- Риск циркулярности: если ИИ-тулинг разгоняет внедрение TypeScript, то «TS массовый» и «модель любит TS» перестают быть независимыми

6. Что это значит и что дальше

Честный заголовок на сегодня

Чистое измерение пока **не показывает** того, что предсказывали анекдоты, при `n = 1`. Это не «гипотеза опровергнута» и не «Ruby в порядке, расходимся».

Осталось незакрытым:

- `n = 1` → нужен полный прогон (10–100 тикетов на язык)
- Цена и скорость — косвенный прокси, не прямая проверка гипотезы
- Гейт `smoke` гнался в режиме `info`, не `block`

7. Выводы для зала

Выводы для зала

- Буксует агент на Ruby — прежде чем винить язык, посмотрите на свой цикл: сколько уходит на модель, сколько на пересборку и тесты между итерациями
- «Python-коллега, у которого всё просто работает» — возможно, меряет не язык, а свой стенд
- Единственный барьер, который агент не обходит изнутри, — внешняя изоляция уровня ядра ОС
- Доверяйте методике, а не первому прогону — особенно если он подтверждает то, что вы и так думали

Спасибо

Вопросы



иллюстрация